Recognising without recalling:

a small recurrent memory recognises long after it stops recalling

Ivan Rodkin

benchmark: `irodkin/recognition-memory`

Takeaway

A memory can tell whether an input is familiar long after it can no longer reproduce it.

- Shrinking a GDN-2 state, **exact recall fails first**: at $16{\times}16$, $N{=}256$ nothing is copied exactly, yet both readouts score acc9 ≥ 0.996.
- At $16{\times}4$, $N{=}256$ the memory **trained for recognition** is ahead: acc9 **0.88** [0.74–1.00] vs. **0.45** [0.27–0.63] for surprise (chance 0.11).
- Training it needs a **length curriculum**, and at small states it is **not reproducible**: the same seed can learn or collapse.

Hypothesis

Memory need not *reproduce* the past. It can instead *recognise* whether an input is consistent with it, using low **surprise** as the familiarity signal. Checking is easier than generating, so recognition should need **less capacity** than recall.

Benchmark: one sequence, eight near-copies

Uniform i.i.d. tokens, vocabulary 16 (4 bits each), $N = 4 \ldots 256$. Each comes with **8 corrupted copies**: one forced change plus $p = \frac{1}{N-1}$ elsewhere, so **2 changes on average**.

true	3	a	0	f	7	7	c	1	9	e	4	2	b	5	d	8
copy	3	a	0	f	7	2	c	1	9	e	4	2	b	5	d	8

Task. Score the true sequence and its 8 copies.
acc9: true one ranked first (chance $1/9$). **pairwise**: true beats one copy (chance $1/2$).

Capacity, in theory. Recall needs all $4N$ bits (1,024 at $N{=}256$). Recognition needs a few: keeping only the token sum mod 16 (4 bits) gives acc9 0.85; adding a position-weighted sum (8 bits) gives 0.965, at *every* N.

Model: one network, two readouts

2-layer Gated DeltaNet-2 (`fla`), width 128, one head. The only memory is the recurrent state, $d_k{\times}d_v$ per layer: $128{\times}128$ (16,384 numbers) down to $16{\times}4$ (64).

state $S_t \in \mathbb{R}^{d_k \times d_v}$ carries x forward

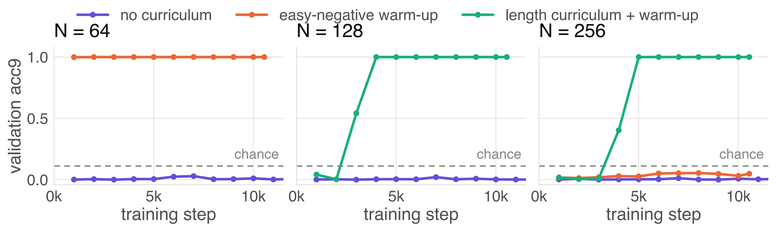


Recall model
trained to copy x (next-token CE)
score $= -$surprise
$= \sum_j \log p(c_j \mid x, c_{<j})$

Recognition model
trained on "is $c = x$?"
(BCE, or ranking loss)
score = logit at the last token

1 Full-size state: recall is easy, recognition needs a curriculum

At $128{\times}128$ the **recall model** copies every length almost perfectly (exact copy 0.988 at $N{=}256$), and its surprise ranks the true sequence first in *every* test sample. The **recognition model**, trained directly, fails from $N{=}64$ on: its loss sits at $\ln 2$ (a constant output) for all 35k steps. The same network learns recall in $<1{,}000$ steps, so this is an **optimisation** failure, not a capacity limit.
Length curriculum: for the first half of training, use prefixes $16, 32, \ldots, N$ (equal time each), plus an **easy-negative warm-up** (heavily corrupted negatives that fade out); then full length.

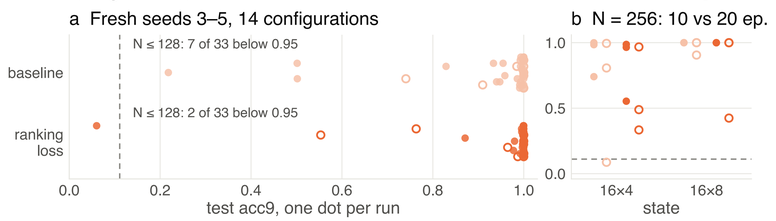


Validation acc9 (full-length sequences), $128{\times}128$ recognition model. Curriculum runs: 3 epochs; no-curriculum runs: 10 epochs (first 11k steps shown).

- **Length curriculum**: acc9 $= 1.000$ at $N{=}128$ and 256. The warm-up alone fixes only $N{=}64$.
- With it, **all 72** recognition runs of the sweep and seeds train; none ends at $\ln 2$.

2 Small states: training is a coin flip

The same code and seed gave acc9 **0.871** in the sweep and **0.058** in a rerun on another GPU model (kernels aren't bit-identical). Each small-state run is one draw from a two-humped distribution. Tried: a **ranking loss**, loss $= \mathrm{softplus}(s_{\text{neg}} - s_{\text{true}})$ on the same last-token score, curriculum from length 4.

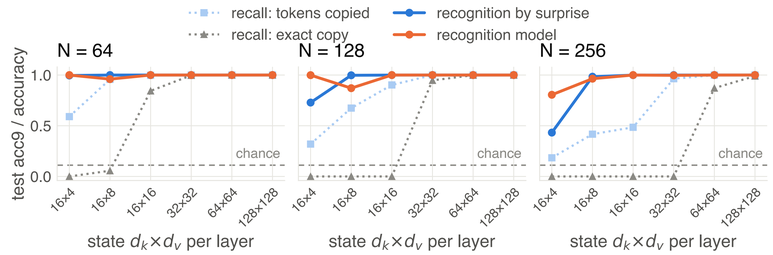


Recognition model, baseline and **ranking loss**, 3 runs per configuration ($16{\times}4$–$16{\times}16$). (a) hollow: $N{=}256$. (b) filled: 10 epochs; hollow: 20, more curriculum time on long prefixes.

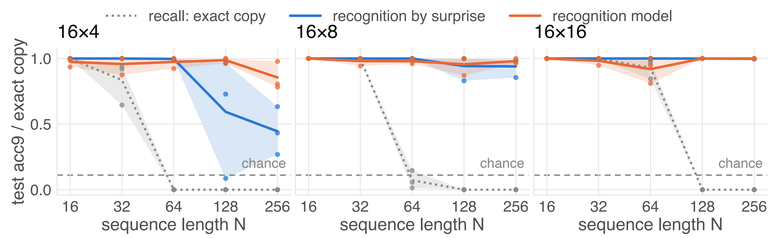
- Up to $N{=}128$ the **ranking loss** fails less (2 vs. 7 of 33 runs below 0.95; $p{=}0.07$): suggestive, not significant. It also collapses (0.06).
- At $N{=}256$ it is worse (0.92 vs. 0.96), and 20 epochs hurt both: $16{\times}4$ 0.63/0.60 vs. 0.91/0.84.

3 Shrinking the state: recall goes first

6 state sizes ($16{\times}4$ to $128{\times}128$) $\times$ 7 lengths $\times$ 2 objectives, 10 epochs each, tested on 100k sequences; 3 seeds at the three smallest states. Recognition model: BCE + length curriculum.



Seed 0, test acc9: **recall model's surprise**, **recognition model**. Dotted: recall. $N \leq 32$: all ≥ 0.87.



Three smallest states, seeds 0–2: mean (line), min–max (band), each seed (dots).

- **Recognition outlasts recall.** At $16{\times}16$, $N{=}256$ the recall model keeps ~half the bits (≥ 536 of 1,024) and copies nothing exactly, yet both readouts score acc9 ≥ 0.996.
- **Smallest state: the recognition model is ahead.**

$16{\times}4$	recall tokens	surprise, 3 runs	recognition, 6 runs
$N{=}128$	0.30	0.59 [0.09–0.97]	**0.98** [0.95–1.00]
$N{=}256$	0.20	0.45 [0.27–0.63]	**0.88** [0.74–1.00]

  At $N{=}256$ its worst run (0.74) beats surprise's best (0.63). Few runs per cell, but large margins.
- Near capacity, **surprise** depends on what the memory chose to keep: 0.97, 0.73, 0.09 across seeds at $N{=}128$.
- At $16{\times}8$ the two are about even ($N{=}256$: 0.94 vs. 0.98).

Next

- **More runs per cell** (6–10, both readouts) to measure failure rates; find why runs collapse.
- **Harder benchmarks** where checksums fail; K stored sequences per context.

3 seeds at $16{\times}4$–$16{\times}16$ for $N \geq 16$ (6 recognition runs in the $16{\times}4$ table), seed 0 elsewhere. The three smallest states shrink d_v only (fla kernels need $d_k \geq 16$); across them the parameter count changes by 4%.